LEKCJA 05

Lekcja 05: Kolekcje I – listy i łańcuchy znaków w Data Science

#lekcja #datascience #python #kolekcje #listy #stringi

Spis treści

1 Listy w Pythonie – podstawy i indeksowanie
2 Metody list i typowe wzorce
3 Stringi – podstawy i indeksowanie
4 Metody stringów – czyszczenie, transformacja, walidacja
5 Zaawansowane formatowanie f-stringów
6 🌱 Aspekt środowiskowy
7 📊 Praca z prawdziwymi danymi
8 🖊️ Zadania do samodzielnej pracy
9 📚 Podsumowanie
10 📖 Dodatkowe zasoby

W tej lekcji poznasz dwie fundamentalne struktury danych w Pythonie: **listy** i **łańcuchy znaków** (stringi). Nauczysz się je tworzyć, indeksować, wycinać (slicing) oraz modyfikować za pomocą wbudowanych metod. Zrozumiesz kluczową różnicę między **mutowalnością** list a **niemutowalnością** stringów oraz poznasz zaawansowane możliwości formatowania z użyciem f-stringów.

W tej lekcji korzystamy wyłącznie z wiedzy z Lekcji 1–4 (zmienne, typy, operatory, f-stringi) oraz nowych koncepcji list i stringów. Pętle, warunki, słowniki i krotki poznasz w kolejnych lekcjach.

---

1. Listy w Pythonie – podstawy i indeksowanie

> 📚 **Definicja**
>
> **Lista** to uporządkowana, **mutowalna** kolekcja elementów, która może przechowywać wartości różnych typów. W Data Science listy są używane do przechowywania pomiarów, nazw kolumn, wyników eksperymentów czy parametrów modeli.

Wprowadzenie

Lista to jedna z najczęściej używanych struktur danych w Pythonie. Tworzymy ją za pomocą nawiasów kwadratowych `[]`, a elementy oddzielamy przecinkami. Kolejność ma znaczenie: każdy element ma swój numer pozycji (indeks). Listy są mutowalne, co znaczy, że po utworzeniu możemy zmieniać ich zawartość — dodawać, usuwać, modyfikować elementy.

W praktyce Data Science listy służą do:

- przechowywania serii pomiarów i wyników obliczeń,
- przechowywania nazw kolumn zestawu danych,
- budowania kolekcji wyników eksperymentów przed analizą,
- reprezentacji macierzy (lista list) — np. confusion matrix.

Przykład 1: Tworzenie list i podstawowe operacje

```
# Lista pomiarów temperatury (°C) z 7 kolejnych dni
temperatury = [22.5, 23.1, 21.8, 24.3, 22.9, 23.5, 22.0]
print(f"Pomiary temperatury: {temperatury}")
print(f"Liczba pomiarów: {len(temperatury)}")

# Lista nazw kolumn datasetu
kolumny = ['temperatura', 'wilgotnosc', 'cisnienie', 'predkosc_wiatru']
print(f"Kolumny datasetu: {kolumny}")
```

```
# Lista może zawierać różne typy danych
mixed_data = [42, 3.14, "Data Science", True, None]
print(f"Lista z różnymi typami: {mixed_data}")

# Pusta lista
wyniki = []
print(f"Pusta lista: {wyniki}, długość: {len(wyniki)}")

# Lista wypełniona powtarzającą się wartością
zera = [0.0] * 5
print(f"Pięć zer: {zera}")

# Wbudowane funkcje agregujące
print(f"Najniższa temperatura: {min(temperatury)} °C")
print(f"Najwyższa temperatura: {max(temperatury)} °C")
print(f"Suma: {sum(temperatury)} °C")
print(f"Średnia: {sum(temperatury) / len(temperatury):.2f} °C")
```

Wyjaśnienie: Listę tworzymy przez nawiasy kwadratowe `[]`. Funkcja `len()` zwraca liczbę elementów, a wbudowane `min()`, `max()`, `sum()` działają na listach liczb. Operator `*` powtarza elementy listy — wygodny sposób na utworzenie listy wypełnionej jedną wartością.

Przykład 2: Indeksowanie i wycinki (slicing)

```
# Dane sprzedaży (w tysiącach) dla 12 miesięcy
sprzedaz = [120, 135, 142, 138, 155, 168, 175, 182, 190, 185, 195, 210]

# Indeksowanie dodatnie (od 0)
print(f"Styczeń (indeks 0): {sprzedaz[0]}")
print(f"Marzec (indeks 2): {sprzedaz[2]}")

# Indeksowanie ujemne (od końca, -1 = ostatni)
print(f"Grudzień (indeks -1): {sprzedaz[-1]}")
print(f"Listopad (indeks -2): {sprzedaz[-2]}")

# Slicing [start:stop] – stop wyłącznie
q1 = sprzedaz[0:3]     # miesiące 0, 1, 2 → styczeń, luty, marzec
print(f"Q1 (pierwszy kwartał): {q1}")

q4 = sprzedaz[9:12]    # miesiące 9, 10, 11 → październik, listopad, grudzień
print(f"Q4 (czwarty kwartał): {q4}")

# Skrócone formy slicingu
```

```
poczatek_roku = sprzedaz[:6]   # od początku do indeksu 6 (wyłącznie)
koniec_roku   = sprzedaz[6:]   # od indeksu 6 do końca
print(f"Pierwsze 6 miesięcy: {poczatek_roku}")
print(f"Drugie 6 miesięcy: {koniec_roku}")

# Slicing z ujemnym stopem
bez_grudnia = sprzedaz[:-1]
print(f"Bez ostatniego miesiąca: {bez_grudnia}")

# Slicing ze skokiem [start:stop:step]
co_drugi = sprzedaz[::2]     # co drugi element
print(f"Co drugi miesiąc: {co_drugi}")

odwrocona = sprzedaz[::-1]   # krok -1 → odwrócenie
print(f"Kolejność odwrócona: {odwrocona}")

# Mutowalność – modyfikacja przez indeks
sprzedaz[0] = 125   # korekta stycznia
print(f"Po korekcie stycznia: {sprzedaz}")

# Modyfikacja całego fragmentu
sprzedaz[0:3] = [130, 140, 150]
print(f"Po podmianie Q1: {sprzedaz}")
```

Wyjaśnienie: Indeksowanie zaczyna się od 0. Slicing `[start:stop]` zwraca nową listę (stop jest wyłączny). Możemy pominąć `start` lub `stop`, a także podać ujemny krok, aby odwrócić listę. Ponieważ listy są mutowalne, możemy zastępować pojedyncze elementy lub całe fragmenty.

Przykład 3: Listy list – struktury dwuwymiarowe

```
# Confusion matrix 2x2 dla klasyfikacji binarnej
confusion_matrix = [
    [50, 5],   # wiersz 0: TN (True Negative), FP (False Positive)
    [3, 42]    # wiersz 1: FN (False Negative), TP (True Positive)
]
print(f"Confusion Matrix:")
print(f"  {confusion_matrix[0]}")
print(f"  {confusion_matrix[1]}")

# Indeksowanie dwuwymiarowe: [wiersz][kolumna]
tn = confusion_matrix[0][0]
fp = confusion_matrix[0][1]
fn = confusion_matrix[1][0]
```

```
tp = confusion_matrix[1][1]

print(f"TN = {tn}, FP = {fp}, FN = {fn}, TP = {tp}")

# Obliczenia bezpośrednio na elementach listy
accuracy = (tp + tn) / (tp + tn + fp + fn)
precision = tp / (tp + fp)
recall = tp / (tp + fn)

print(f"Accuracy : {accuracy:.2%}")
print(f"Precision: {precision:.2%}")
print(f"Recall   : {recall:.2%}")

# Macierz cech dla 3 próbek × 4 cech
X = [
    [5.1, 3.5, 1.4, 0.2],
    [7.0, 3.2, 4.7, 1.4],
    [6.3, 3.3, 6.0, 2.5]
]
print(f"\nCecha 0 próbki 0: {X[0][0]}")
print(f"Cała próbka 2: {X[2]}")
```

Wyjaśnienie: Lista list to naturalna reprezentacja danych dwuwymiarowych. Dostęp do elementu wymaga dwóch indeksów: `macierz[wiersz][kolumna]`. Taka struktura jest fundamentem dla NumPy i Pandas, które poznamy w kolejnych modułach.

Wizualizacja: indeksowanie listy

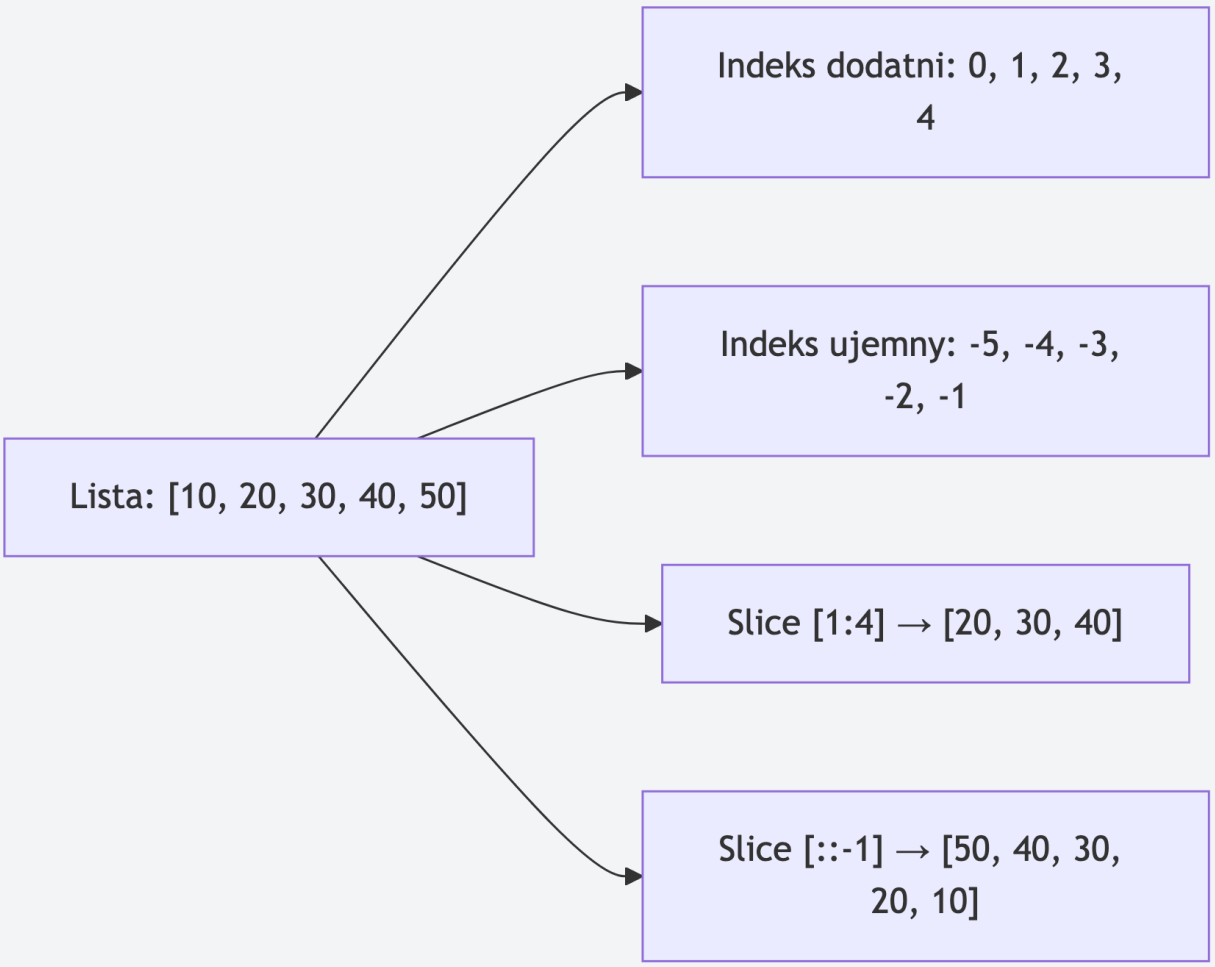


Diagram 1

> 💡 **Wskazówka**
>
> W Data Science często używamy slicingu, aby podzielić dane na zbiory treningowy i testowy: `train = dane[:800]` oraz `test = dane[800:]`.

> ⚠️ **Uwaga**
>
> Slicing poza zakresem nie zgłasza błędu (zwraca pustą listę lub listę krótszą), ale pojedynczy indeks poza zakresem powoduje `IndexError`. Zawsze sprawdzaj `len(lista)` przed indeksowaniem danych z niepewnego źródła.

---

2. Metody list i typowe wzorce

> 📚 **Definicja**
>
> **Metody list** to wbudowane funkcje, które pozwalają efektywnie manipulować danymi: dodawać, usuwać, sortować i przeszukiwać elementy. Wywołujemy je składnią `lista.metoda(argumenty)`.

Wprowadzenie

Listy posiadają bogaty zestaw metod wbudowanych, dzięki którym nie musimy pisać kodu modyfikującego elementy ręcznie. W Data Science najczęściej używamy metod do zbierania wyników eksperymentów, porządkowania danych i sprawdzania obecności wartości.

Przykład 1: Dodawanie elementów – append, extend, insert

```
# Zbieramy wyniki accuracy z kolejnych eksperymentów
experiment_results = []

# append() – dodaje JEDEN element na końcu (operacja O(1))
experiment_results.append(0.82)
experiment_results.append(0.85)
experiment_results.append(0.83)
print(f"Po 3 append'ach: {experiment_results}")

# extend() – dodaje WIELE elementów z innej listy
nowe_wyniki = [0.86, 0.84, 0.87]
experiment_results.extend(nowe_wyniki)
print(f"Po extend: {experiment_results}")
```

```
# Różnica: append wkłada CAŁĄ listę jako jeden element
lista_a = [1, 2, 3]
lista_b = [1, 2, 3]

lista_a.append([4, 5])   # [4, 5] staje się jednym elementem
lista_b.extend([4, 5])   # 4 i 5 dodane pojedynczo

print(f"Po append([4,5]): {lista_a}")   # [1, 2, 3, [4, 5]]
print(f"Po extend([4,5]): {lista_b}")   # [1, 2, 3, 4, 5]

# insert(indeks, wartość) – wstawia na konkretnej pozycji
features = ['wiek', 'dochod', 'wyksztalcenie']
features.insert(1, 'plec')   # wstawiamy 'plec' na indeks 1
print(f"Po insert(1, 'plec'): {features}")

# Operator + tworzy NOWĄ listę (oryginał niezmieniony)
baza = ['id', 'name']
rozszerzona = baza + ['email', 'city']
print(f"Baza: {baza}")
print(f"Rozszerzona: {rozszerzona}")

# Operator * powtarza elementy listy
placeholder = ['NA'] * 4
print(f"Placeholdery: {placeholder}")
```

Wyjaśnienie: `append()` dodaje pojedynczy element na końcu (bardzo szybkie), `extend()` rozszerza listę o wiele elementów, `insert()` wstawia w dowolnym miejscu (wolniejsze, bo przesuwa elementy). Operator `+` tworzy nową listę — jest czytelny, ale mniej efektywny niż `extend()` przy dużych zbiorach.

Przykład 2: Usuwanie elementów – remove, pop, clear, del

```
# Seria pomiarów z kilkoma wartościami odstającymi
measurements = [23.5, 24.1, 23.8, 150.0, 24.3, 23.9, -10.5, 24.0]
print(f"Oryginalne pomiary: {measurements}")

# remove(wartość) – usuwa PIERWSZE wystąpienie wartości
measurements.remove(150.0)
print(f"Po remove(150.0): {measurements}")

# pop(indeks) – usuwa i ZWRACA element
outlier = measurements.pop(5)   # indeks 5 to teraz -10.5
print(f"Usunięta wartość: {outlier}")
```

```
print(f"Po pop(5): {measurements}")

# pop() bez argumentu usuwa ostatni element
ostatnia = measurements.pop()
print(f"Usunięta ostatnia: {ostatnia}")
print(f"Aktualna lista: {measurements}")

# del – instrukcja usuwająca przez indeks
del measurements[0]
print(f"Po del[0]: {measurements}")

# del ze slicingiem – usuwa zakres
kopia = [10, 20, 30, 40, 50, 60, 70]
del kopia[2:5]
print(f"Po del[2:5]: {kopia}")

# Przypisanie pustego slice też usuwa
druga_kopia = [10, 20, 30, 40, 50, 60, 70]
druga_kopia[2:5] = []
print(f"Po [2:5] = []: {druga_kopia}")

# clear() – czyści całą listę
temp = [1, 2, 3, 4, 5]
temp.clear()
print(f"Po clear(): {temp}")
```

Wyjaśnienie: `remove()` szuka po wartości (zgłasza `ValueError`, gdy wartość nie istnieje), `pop()` działa po indeksie i zwraca usunięty element (przydatne, gdy potrzebujemy wartości). `del` to instrukcja (nie metoda), której można użyć ze slicingiem. `clear()` opróżnia listę w miejscu.

Przykład 3: Sortowanie i odwracanie – sort, sorted, reverse

```
# Wyniki accuracy z różnych modeli
model_scores = [0.78, 0.92, 0.85, 0.88, 0.81, 0.95, 0.79]
print(f"Oryginalne wyniki: {model_scores}")

# sort() – modyfikuje listę IN-PLACE
model_scores.sort()
print(f"Po sort() rosnąco: {model_scores}")

model_scores.sort(reverse=True)
print(f"Po sort(reverse=True): {model_scores}")
```

```python
# Top 3 wyniki → slicing posortowanej listy
top_3 = model_scores[:3]
print(f"Top 3 modele: {top_3}")

# sorted() – zwraca NOWĄ listę (oryginał niezmieniony)
scores_original = [0.78, 0.92, 0.85, 0.88]
scores_sorted = sorted(scores_original)
print(f"Oryginalna: {scores_original}")
print(f"Sorted:     {scores_sorted}")

# Sortowanie stringów (alfabetyczne)
modele = ['XGBoost', 'RandomForest', 'LogisticRegression', 'SVM', 'KNN']
modele.sort()
print(f"Alfabetycznie: {modele}")

# reverse() – odwraca IN-PLACE (niezależnie od sortowania)
liczby = [1, 2, 3, 4, 5]
liczby.reverse()
print(f"Po reverse(): {liczby}")

# sorted() z parametrem reverse
alfabet_odwrotnie = sorted(['banan', 'jabłko', 'czereśnia'], reverse=True)
print(f"Owoce malejąco: {alfabet_odwrotnie}")
```

Wyjaśnienie: `sort()` jest szybsze i oszczędniejsze pamięciowo (modyfikuje istniejącą listę), ale niszczy oryginalną kolejność. `sorted()` jest bezpieczniejsze (oryginał zostaje nietknięty), ale wymaga dodatkowej pamięci. W obu przypadkach `reverse=True` daje porządek malejący.

Przykład 4: Przeszukiwanie – index, count, in

```python
# Wyniki predykcji z modelu klasyfikacji
predictions = ['cat', 'dog', 'cat', 'bird', 'dog', 'cat', 'dog', 'cat', 'bird']
print(f"Predykcje: {predictions}")

# count() – liczba wystąpień wartości
cat_count  = predictions.count('cat')
dog_count  = predictions.count('dog')
bird_count = predictions.count('bird')
print(f"cat: {cat_count}, dog: {dog_count}, bird: {bird_count}")

# index(wartość) – indeks PIERWSZEGO wystąpienia
first_bird = predictions.index('bird')
print(f"Pierwszy 'bird' na pozycji: {first_bird}")
```

```python
# index() z dodatkowym zakresem startowym
second_bird = predictions.index('bird', first_bird + 1)
print(f"Drugi 'bird' na pozycji: {second_bird}")

# Operator `in` / `not in` – zwraca bezpośrednio True/False
has_cat      = 'cat' in predictions
has_elephant = 'elephant' in predictions
no_elephant  = 'elephant' not in predictions

print(f"Model przewidział kota? {has_cat}")
print(f"Model przewidział słonia? {has_elephant}")
print(f"Słoń nieobecny? {no_elephant}")

# Porównanie rozkładu klas bez iteracji – wystarczy count
total = len(predictions)
cat_pct  = cat_count  / total * 100
dog_pct  = dog_count  / total * 100
bird_pct = bird_count / total * 100

print(f"Rozkład: cat {cat_pct:.1f}%, dog {dog_pct:.1f}%, bird {bird_pct:.1f}%")
```

Wyjaśnienie: `count()` działa szybciej niż ręczne liczenie. `index()` zgłasza `ValueError`, jeśli wartości nie ma — dlatego wygodniej najpierw sprawdzić obecność przez operator `in`. Operator `in` zwraca wartość typu `bool`, którą można bezpośrednio przypisać do zmiennej lub wypisać.

Wizualizacja: operacje na listach

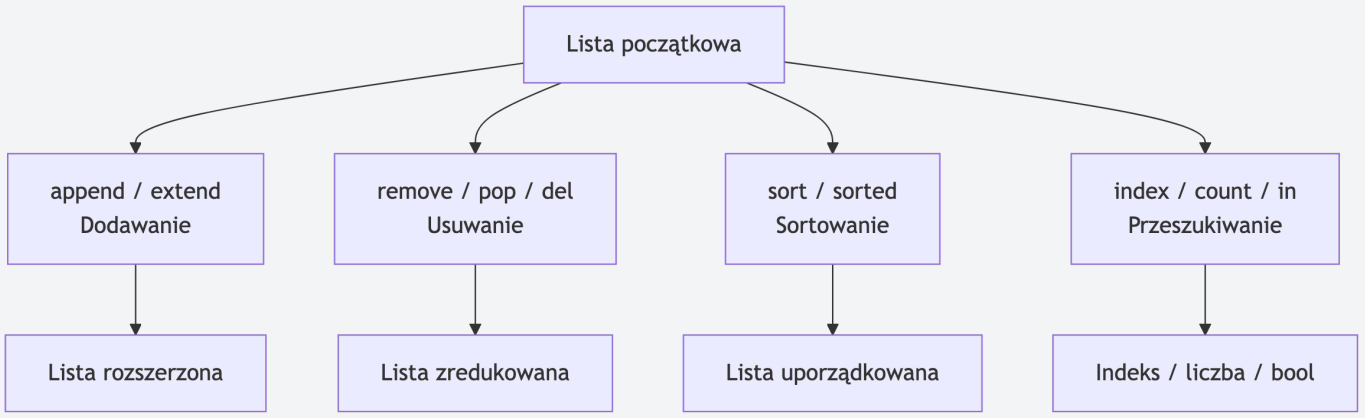


Diagram 2

💡 **Wskazówka**

`append()` jest fundamentalną metodą w Data Science — używamy jej do zbierania wyników eksperymentów, predykcji, metryk. Preferuj `append()` zamiast `lista = lista + [element]` — jest szybszy i nie tworzy niepotrzebnych kopii.

⚠️ **Uwaga**

Metody takie jak `sort()`, `reverse()`, `append()` zwracają `None`, a nie listę. Częsty błąd: `nowa = lista.sort()` spowoduje, że `nowa` będzie `None`. Używaj `sorted()`, jeśli potrzebujesz nowej listy jako wyniku.

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

3. Stringi – podstawy i indeksowanie

> 📚 **Definicja**
>
> **String** (`str`) to uporządkowana, **niemutowalna** sekwencja znaków. W Data Science stringi reprezentują kategorie, nazwy kolumn, etykiety klas, linie logów, ścieżki do plików, komunikaty błędów.

Wprowadzenie

Stringi tworzymy za pomocą cudzysłowów: pojedynczych `'...'`, podwójnych `"..."` lub potrójnych `"""..."""` (do tekstów wieloliniowych). Wszystkie są równoważne funkcjonalnie — wybór zależy od tego, jakie znaki pojawiają się w środku (np. cudzysłowy) i od stylu projektu. Kluczową cechą stringów jest **niemutowalność**: nie możemy zmienić pojedynczego znaku — każda "modyfikacja" tworzy nowy string.

Przykład 1: Tworzenie stringów i f-stringi

```
# Różne formy tworzenia
single = 'Data Science'
double = "Machine Learning"
multi  = """Python
to mój ulubiony
język programowania"""

print(single)
print(double)
print(multi)

# Cudzysłowy w środku stringa
cytat = "Mówi: 'Python jest czytelny'"
cytat_alt = 'Ona powiedziała "tak"'
print(cytat)
print(cytat_alt)

# Znaki specjalne
z_nowa_linia = "Linia 1\nLinia 2\nLinia 3"
print(z_nowa_linia)

z_tabulatorem = "Kolumna1\tKolumna2\tKolumna3"
```

```
print(z_tabulatorem)

# Raw string – backslash traktowany dosłownie
sciezka = r"C:\Users\data\dataset.csv"
print(sciezka)

# Konkatenacja operatorem +
imie = "Anna"
nazwisko = "Kowalska"
pelne_imie = imie + " " + nazwisko
print(pelne_imie)

# Powtarzanie operatorem *
separator = "-" * 40
print(separator)
print("NAGŁÓWEK RAPORTU")
print(separator)

# f-string (najwygodniejsza metoda formatowania – z Lekcji 4)
model = "RandomForest"
accuracy = 0.8756
print(f"Model: {model}, accuracy: {accuracy:.2%}")

# Wieloliniowy f-string
raport = f"""
Model:     {model}
Accuracy:  {accuracy:.2%}
"""
print(raport)

# len() działa na stringu identycznie jak na liście
print(f"Liczba znaków w 'Data Science': {len(single)}")
```

Wyjaśnienie: Potrójne cudzysłowy są wygodne dla tekstów wielolinijkowych (zachowują nowe linie i wcięcia). Raw stringi (`r"..."`) przydają się przy ścieżkach Windows i wyrażeniach regularnych. F-string to domyślna metoda formatowania w nowoczesnym kodzie Pythona.

Przykład 2: Indeksowanie i slicing stringów

```
# String zachowuje się w indeksowaniu jak lista znaków
filename = "sales_data_2024_Q1.csv"

# Indeksowanie pojedynczych znaków
```

```
print(f"Pierwszy znak : '{filename[0]}'")    # 's'
print(f"Drugi znak    : '{filename[1]}'")    # 'a'
print(f"Ostatni znak  : '{filename[-1]}'")   # 'v'
print(f"Przedostatni  : '{filename[-2]}'")   # 's'

# Slicing do ekstrakcji fragmentów
rozszerzenie = filename[-4:]   # ".csv"
nazwa_bez_ext = filename[:-4]  # "sales_data_2024_Q1"
rok = filename[11:15]          # "2024"
kwartal = filename[16:18]      # "Q1"

print(f"Rozszerzenie: '{rozszerzenie}'")
print(f"Nazwa bez rozszerzenia: '{nazwa_bez_ext}'")
print(f"Rok: '{rok}', kwartał: '{kwartal}'")

# len() zwraca liczbę znaków
print(f"Długość nazwy: {len(filename)} znaków")

# Operator `in` sprawdza obecność podstringa
has_csv  = '.csv'  in filename
has_json = '.json' in filename
print(f"Jest .csv? {has_csv}")
print(f"Jest .json? {has_json}")

# Niemutowalność — poniższe spowoduje TypeError
# filename[0] = 'S'

# Zamiast tego tworzymy nowy string
pierwsza_wielka = filename[0].upper() + filename[1:]
print(f"Pierwsza litera wielka: {pierwsza_wielka}")
```

Wyjaśnienie: Stringi obsługują te same operacje indeksowania i slicingu co listy. Kluczowa różnica: stringów **nie można** modyfikować w miejscu. Każda "zmiana" (np. podmiana pierwszej litery na wielką) tworzy nowy obiekt. Dla czyszczenia danych jest to ograniczenie, które rozwiązujemy przez dedykowane metody (następna sekcja).

Wizualizacja: string jako sekwencja znaków

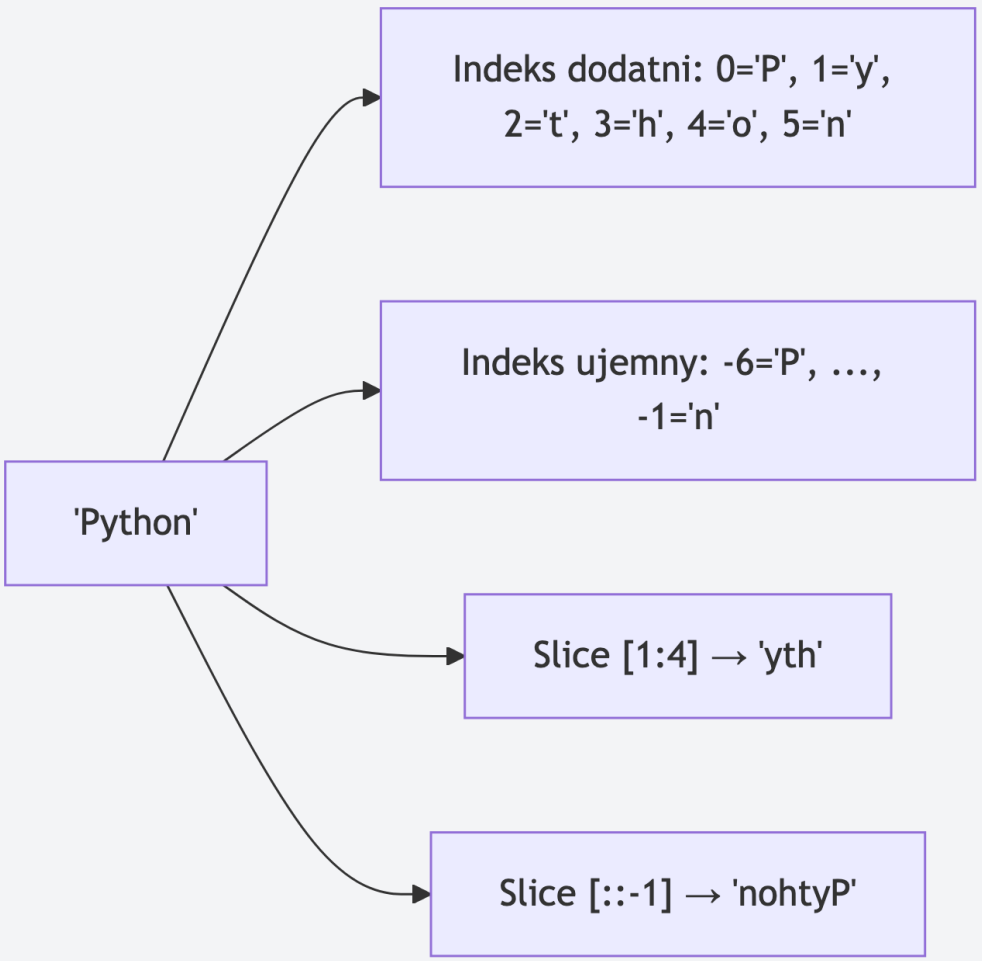


Diagram 3

> 💡 **Wskazówka**
>
> Stringi i listy współdzielą bardzo podobny interfejs (indeksowanie, slicing, `len`, `in`, `+`, `*`). Ta spójność to celowa cecha Pythona — łatwiej jest się uczyć nowych struktur, gdy działają "w znajomy sposób".

---

4. Metody stringów – czyszczenie, transformacja, walidacja

Wprowadzenie

Dane tekstowe pochodzące z plików CSV, logów, API czy formularzy użytkownika są niemal zawsze "brudne": mają zbędne spacje, niejednolity rozmiar liter, różne separatory, puste wartości. Metody stringów pozwalają szybko doprowadzić je do postaci analitycznej.

Przykład 1: Transformacje – strip, upper, lower, title, replace

```
sample = "  Machine Learning Model  "

# Usuwanie białych znaków
print(f"strip()  : '{sample.strip()}'")    # z obu stron
print(f"lstrip() : '{sample.lstrip()}'")   # tylko z lewej
print(f"rstrip() : '{sample.rstrip()}'")   # tylko z prawej

# Zmiana wielkości liter
print(f"upper()      : '{sample.upper()}'")
print(f"lower()      : '{sample.lower()}'")
print(f"title()      : '{sample.title()}'")
print(f"capitalize() : '{sample.capitalize()}'")
print(f"swapcase()   : '{sample.swapcase()}'")

# replace(stare, nowe) – tworzy nowy string
zdanie = "Python jest szybki. Python jest czytelny."
print(zdanie.replace("Python", "Pandas"))

# replace z limitem wystąpień
print(zdanie.replace("Python", "Pandas", 1))   # tylko pierwsze wystąpienie

# Czyszczenie JEDNEJ nazwy kolumny – łańcuch metod
raw_col = "  Customer ID  "
clean_col = raw_col.strip().lower().replace(' ', '_')
print(f"'{raw_col}' → '{clean_col}'")

# Czyszczenie pojedynczej kategorii
kategoria_raw = "  ELECTRONICS  "
kategoria_norm = kategoria_raw.strip().lower()
print(f"Znormalizowana kategoria: '{kategoria_norm}'")
```

Wyjaśnienie: `strip()` to najważniejsza metoda przy czyszczeniu danych — usuwa wszystkie białe znaki (spacje, tabulatory, nowe linie) z obu stron. `lower()` / `upper()` ujednolicają wielkość liter, co jest krytyczne przy porównywaniu kategorii. `replace()` podmienia fragmenty tekstu, a dzięki niemutowalności stringów każda operacja jest bezpieczna — oryginał się nie zmienia.

Przykład 2: Walidacja – startswith, endswith, isdigit, isalpha

```
# Sprawdzanie rozszerzenia pliku
filename = "sales_data_2024.csv"
print(f"Jest CSV?   {filename.endswith('.csv')}")
print(f"Jest Excel? {filename.endswith('.xlsx')}")

# Sprawdzanie prefiksu
print(f"Plik sprzedaży? {filename.startswith('sales')}")

# endswith i startswith akceptują też listę w krotce... ale krotki poznamy w L6.
# Na razie możemy sprawdzać pojedynczo i łączyć operatorami logicznymi:
czy_csv  = filename.endswith('.csv')
czy_tsv  = filename.endswith('.tsv')
czy_text = czy_csv or czy_tsv
print(f"Jest plikiem tekstowym (CSV/TSV)? {czy_text}")

# Walidacja pojedynczych wartości
age  = "28"
name = "Alice"
mixed = "Alice28"
spacje_tylko = "   "

print(f"'{age}'   isdigit : {age.isdigit()}")      # True
print(f"'{name}'  isalpha : {name.isalpha()}")     # True
print(f"'{mixed}' isalpha : {mixed.isalpha()}")    # False (zawiera cyfry)
print(f"'{mixed}' isalnum : {mixed.isalnum()}")    # True
print(f"'{spacje_tylko}' isspace : {spacje_tylko.isspace()}")   # True

# Praktyczny przykład – klasyfikacja pojedynczego pola
pole_uzytkownika = "  42  "
oczyszczone = pole_uzytkownika.strip()
czy_liczba = oczyszczone.isdigit()
print(f"'{pole_uzytkownika}' po strip: '{oczyszczone}', to liczba? {czy_liczba}")

# find – zwraca indeks podstringa lub -1, jeśli go nie ma
email = "user@example.com"
at_position = email.find('@')
```

```
print(f"'@' w emailu na pozycji: {at_position}")

# count dla stringów – liczy wystąpienia podstringa
tekst = "banana banana banana"
print(f"Liczba 'banan': {tekst.count('banan')}")
```

Wyjaśnienie: `startswith()` / `endswith()` idealnie pasują do sprawdzania rozszerzeń plików i prefiksów. Metody `is*()` zwracają `bool`, więc wynik można bezpośrednio przypisać do zmiennej. `find()` to bezpieczniejsza wersja `index()` dla stringów — zwraca `-1` zamiast zgłaszać wyjątek.

Przykład 3: Parsowanie – split i join

```
# split(separator) – dzieli string na listę
linia_csv = "Alice,28,Data Scientist,75000"
pola = linia_csv.split(',')
print(f"Po split(',') : {pola}")

# Dostęp do pól przez indeks + konwersja typu
imie   = pola[0]
wiek   = int(pola[1])
zawod  = pola[2]
pensja = int(pola[3])

print(f"{imie}, {wiek} lat, {zawod}, ${pensja}")

# split bez argumentu – dzieli po dowolnych białych znakach
tekst = "Python   jest\t fajny\nnaprawdę"
slowa = tekst.split()
print(f"Słowa: {slowa}")

# split z limitem – maksymalna liczba podziałów
log = "2024-01-15 10:23:45 ERROR Training failed due to OOM"
fragmenty = log.split(' ', 3)   # max 3 podziały → 4 elementy
print(f"Fragmenty logu: {fragmenty}")

data   = fragmenty[0]
czas   = fragmenty[1]
poziom = fragmenty[2]
msg    = fragmenty[3]
print(f"[{data} {czas}] {poziom}: {msg}")

# Parsowanie ścieżki (bez potrzeby importowania modułów)
sciezka = "/data/processed/2024/sales.csv"
```

```
czesci = sciezka.split('/')
print(f"Czesci: {czesci}")

nazwa_pliku = czesci[-1]           # 'sales.csv'
katalog = '/'.join(czesci[:-1])    # '/data/processed/2024'
print(f"Nazwa pliku: {nazwa_pliku}")
print(f"Katalog:     {katalog}")

# join(lista) – łączy listę stringów w jeden string
kolumny = ['id', 'name', 'email', 'city']

csv_header = ','.join(kolumny)
print(f"Nagłówek CSV: {csv_header}")

sql_fields = ', '.join(kolumny)
sql = f"SELECT {sql_fields} FROM customers"
print(f"SQL: {sql}")

# Raport jako wiersze oddzielone \n
wiersze = [
    "Model: RandomForest",
    "Accuracy: 87.56%",
    "Precision: 84.32%",
    "Recall: 89.01%"
]
raport = '\n'.join(wiersze)
print("--- RAPORT ---")
print(raport)

# Budowanie ścieżki (bez modułu os)
kawalki = ['data', 'processed', 'sales_2024.csv']
pelna_sciezka = '/'.join(kawalki)
print(f"Pełna ścieżka: {pelna_sciezka}")
```

Wyjaśnienie: `split()` zwraca listę — idealne do parsowania CSV, logów, ścieżek. Parametr `maxsplit` ogranicza liczbę podziałów, co jest przydatne, gdy chcemy zostawić "resztę" w ostatnim polu (np. wiadomość błędu z logu, która sama może zawierać spacje). `join()` jest operacją odwrotną i jednocześnie NAJEFEKTYWNIEJSZYM sposobem budowania długich tekstów z wielu fragmentów.

Wizualizacja: przepływ operacji na stringach

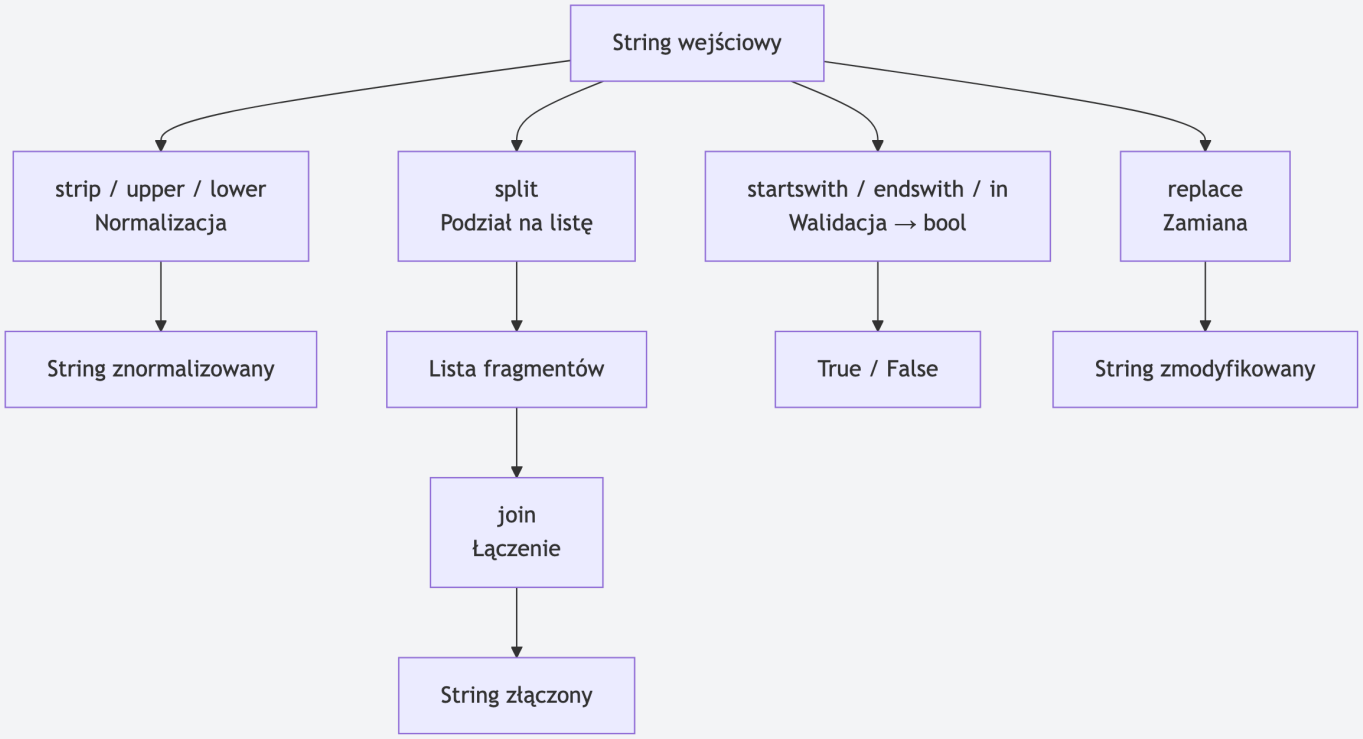


Diagram 4

> 💡 **Wskazówka**
>
> **Best practice:** zawsze normalizuj dane tekstowe przed porównywaniem lub porównuj w spójny sposób — `tekst.strip().lower()` eliminuje 90% problemów z "niedopasowaniem" kategorii typu `"Electronics"` vs `" electronics "`.

> ⚠️ **Uwaga**
>
> `split()` bez argumentów dzieli po dowolnych białych znakach i **ignoruje puste fragmenty** (dobry wybór dla tekstu z niestandardowym odstępem). `split(' ')` dzieli tylko po pojedynczej spacji i **zachowuje puste fragmenty** — co bywa przyczyną trudnych do zdiagnozowania błędów.

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

5. Zaawansowane formatowanie f-stringów

Wprowadzenie

F-stringi wprowadzone w Lekcji 4 mają wbudowany mini-język formatowania, który pozwala precyzyjnie kontrolować szerokość pola, wyrównanie, precyzję liczb, separatory tysięcy i wiele innych aspektów prezentacji danych. W Data Science jest to kluczowe dla tworzenia czytelnych raportów i tabel w konsoli.

Przykład 1: Wyrównanie i szerokość pola

```
model    = "RandomForest"
accuracy = 0.8756

# Szerokość pola – minimalna liczba znaków
print(f"'{model:20}'")     # domyślnie lewo, szerokość 20

# Wyrównanie jawne: < lewo, > prawo, ^ środek
print(f"'{model:<20}'")    # lewo
print(f"'{model:>20}'")    # prawo
print(f"'{model:^20}'")    # środek

# Znak wypełniający przed ^<> (pierwszy znak)
```

```
print(f"'{model:*<20}'")   # 'RandomForest********'
print(f"'{model:*>20}'")   # '********RandomForest'
print(f"'{model:*^20}'")   # '****RandomForest****'

# Wyrównanie liczb
print(f"'{accuracy:>10}'")     # prawo - naturalne dla liczb
print(f"'{accuracy:<10}'")     # lewo
```

Przykład 2: Precyzja liczbowa

```
pi = 3.14159265358979

# .Nf - liczba miejsc po przecinku (float)
print(f"{pi:.2f}")   # 3.14
print(f"{pi:.4f}")   # 3.1416
print(f"{pi:.6f}")   # 3.141593

# .Ne - notacja naukowa
duza_liczba = 1234567.89
print(f"{duza_liczba:.2e}")   # 1.23e+06
print(f"{duza_liczba:.4e}")   # 1.2346e+06

# .N% - procent (mnoży razy 100 i dodaje %)
accuracy = 0.8567
print(f"{accuracy:.2%}")   # 85.67%
print(f"{accuracy:.4%}")   # 85.6700%

# , - separator tysięcy
budzet = 1_234_567
print(f"{budzet:,}")          # 1,234,567
print(f"{budzet:,.2f}")       # 1,234,567.00
print(f"${budzet:,.2f}")      # $1,234,567.00

# + - zawsze wyświetl znak
zmiana = 0.05
print(f"{zmiana:+.2%}")       # +5.00%
print(f"{-zmiana:+.2%}")      # -5.00%
```

Przykład 3: Formatowanie całkowitych (int)

```
i = 5

# Padding z zerami – d mówi "integer", 03 znaczy "szerokość 3, uzupełnij zerami"
print(f"Exp_{i:03d}.log")   # Exp_005.log
print(f"Exp_{i:05d}.log")   # Exp_00005.log

# Hex, oct, bin
n = 255
print(f"Dziesiętnie: {n:d}")
print(f"Szesnastkowo: {n:x}")     # ff
print(f"Szesnastkowo z prefiksem: {n:#x}")  # 0xff
print(f"Binarnie: {n:b}")         # 11111111
```

Przykład 4: Pełen raport – tabela w konsoli

```
# Ręcznie ułożony raport BEZ pętli – 4 modele jako osobne zmienne
m1   = "LogisticRegression"
acc1 = 0.7823
t1   = 1.234

m2   = "RandomForest"
acc2 = 0.8956
t2   = 45.678

m3   = "XGBoost"
acc3 = 0.9234
t3   = 123.456

m4   = "NeuralNetwork"
acc4 = 0.9456
t4   = 567.891

print("=" * 60)
print(f"{'Model':<20} {'Accuracy':>10} {'Time (s)':>12}")
print("=" * 60)
print(f"{m1:<20} {acc1:>10.2%} {t1:>12.2f}")
print(f"{m2:<20} {acc2:>10.2%} {t2:>12.2f}")
print(f"{m3:<20} {acc3:>10.2%} {t3:>12.2f}")
```

```
print(f"{m4:<20} {acc4:>10.2%} {t4:>12.2f}")
print("=" * 60)
```

Wyjaśnienie: Zauważ ciąg `:<20`, `:>10.2%`, `:>12.2f` — to minijęzyk formatowania. Pierwszy znak po `:` może być znakiem wypełniającym (opcjonalnie), potem wyrównanie (`<`, `>`, `^`), szerokość pola, kropka, precyzja i typ (`f`, `e`, `%`, `d`, `x`). Pozwala układać dane w kolumny bez zewnętrznych bibliotek.

> 💡 **Wskazówka**
>
> F-string minijęzyk jest identyczny jak w metodzie `str.format()` — jeśli kiedykolwiek zobaczysz `"{:>10.2f}".format(x)`, wiesz już, co to znaczy.

---

6. 🌱 Aspekt środowiskowy

W tej lekcji zwracamy uwagę na wybór operacji, które zmniejszają zużycie pamięci i CPU — to bezpośrednio przekłada się na ślad węglowy obliczeń Data Science.

Zasada 1: extend() > + dla list

Operator `+` dla list tworzy zupełnie nową listę w pamięci (kopiowanie wszystkich elementów). `extend()` modyfikuje istniejącą listę w miejscu.

```
import sys

# Listy testowe
wyniki = [0.85, 0.87, 0.88, 0.89, 0.90]
wiecej = [0.91, 0.92, 0.93, 0.94, 0.95]

# Podejście kopiujące
nowa_lista = wyniki + wiecej   # nowy obiekt w pamięci

# Podejście in-place
wyniki.extend(wiecej)          # modyfikuje 'wyniki'

print(f"Wynik z '+': {nowa_lista}")
```

```
print(f"Wynik z extend: {wyniki}")
print(f"Rozmiar wyniku: {sys.getsizeof(wyniki)} B")
```

Zasada 2: join() >> konkatenacja + dla stringów

Stringi są niemutowalne, więc każde `s = s + "x"` tworzy nowy obiekt w pamięci. Przy wielu operacjach to zabiera czas i RAM. `join()` alokuje pamięć tylko raz.

```
import time

# Lista 5000 fragmentów
czesci = ['fragment'] * 5000

# Wariant NIEEFEKTYWNY – każde '+' tworzy nowy string
start = time.time()
wynik_bad = ""
wynik_bad = wynik_bad + czesci[0]
wynik_bad = wynik_bad + czesci[1]
wynik_bad = wynik_bad + czesci[2]
# (powtórzone 5000 razy wymagałoby pętli – w L7 zobaczysz, jak to zmierzyć)
# Tu demonstrujemy sam mechanizm:
czas_bad = time.time() - start

# Wariant EFEKTYWNY – jedno wywołanie join
start = time.time()
wynik_good = ''.join(czesci)
czas_good = time.time() - start

print(f"Długość wyniku join: {len(wynik_good)} znaków")
print(f"Czas join: {czas_good:.6f} s")
```

Zasada 3: slicing jest szybszy od kopiowania

Slicing wewnętrznie korzysta z zoptymalizowanych mechanizmów C, więc `lista[a:b]` jest szybsze niż ręczne budowanie kopii. Także przy sortowaniu — `sort()` działa w miejscu (szybsze), `sorted()` tworzy nową listę (bezpieczniejsze, ale więcej pamięci).

Reguły Green IT dla kolekcji:

1. Preferuj `append()` / `extend()` zamiast operatora `+` dla list.

2. Używaj `join()` zamiast konkatenacji `+` w przypadku wielu stringów.
3. Unikaj niepotrzebnego kopiowania: modyfikuj w miejscu, kiedy oryginał nie jest już potrzebny.
4. Czyść duże struktury (`clear()` , `del`), gdy nie są już używane — pomaga garbage collectorowi.
5. Slicing jest szybszy i bardziej czytelny niż ręczne tworzenie kopii.

> ✅ **Sukces**
>
> Efektywny kod = mniej czasu CPU = mniej energii = mniejszy ślad węglowy. W Data Science, gdzie przetwarzamy miliony rekordów, wybór między `+` a `join()` naprawdę wpływa na rachunek za prąd i emisje centrum danych.

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

7. 📊 Praca z prawdziwymi danymi

Parsowanie pojedynczego rekordu z datasetu Iris

Pokażemy, jak za pomocą samych list i stringów możemy sparsować i opisać pojedynczy rekord ze słynnego datasetu Iris. Pełne przetwarzanie wielu rekordów wymaga pętli — wrócimy do tego w Lekcji 7.

```
# Jedna linia w formacie CSV z datasetu Iris
# Kolejność: sepal_length, sepal_width, petal_length, petal_width, species
linia = "5.1,3.5,1.4,0.2,setosa"

# Krok 1: rozdzielenie po przecinku
pola = linia.split(',')
print(f"Pola po split: {pola}")

# Krok 2: konwersja typów
sepal_length = float(pola[0])
sepal_width  = float(pola[1])
petal_length = float(pola[2])
petal_width  = float(pola[3])
species      = pola[4]

# Krok 3: obliczenia na pojedynczym rekordzie
sepal_ratio = sepal_length / sepal_width
petal_ratio = petal_length / petal_width
sepal_area  = sepal_length * sepal_width   # przybliżona "powierzchnia"

# Krok 4: sformatowany raport
print("-" * 40)
print(f"Gatunek: {species}")
print(f"Działka kielicha: {sepal_length} × {sepal_width} cm")
print(f"Płatek:           {petal_length} × {petal_width} cm")
print(f"Proporcja działki:    {sepal_ratio:.2f}")
print(f"Proporcja płatka:     {petal_ratio:.2f}")
print(f"Pseudo-powierzchnia:  {sepal_area:.2f} cm²")
print("-" * 40)
```

Czyszczenie brudnej linii

```
# Ta sama linia, ale "brudna" – tak, jak bywa w realnych danych
brudna = "  5.1 , 3.5 ,1.4,  0.2,  SETOSA  "

# Krok 1: strip globalny (usuwa spacje z początku/końca całej linii)
bez_spacji = brudna.strip()
print(f"Po strip:  '{bez_spacji}'")

# Krok 2: split
pola = bez_spacji.split(',')
print(f"Po split:  {pola}")
```

```
# Krok 3: strip każdego pola indywidualnie (bez pętli – z indeksem)
pole_0 = pola[0].strip()
pole_1 = pola[1].strip()
pole_2 = pola[2].strip()
pole_3 = pola[3].strip()
pole_4 = pola[4].strip().lower()   # kategorię też normalizujemy do małych liter

print(f"Po czyszczeniu: ['{pole_0}', '{pole_1}', '{pole_2}', '{pole_3}', '{pole_4}']")

# Krok 4: konwersja na typy numeryczne
sepal_length = float(pole_0)
sepal_width  = float(pole_1)
petal_length = float(pole_2)
petal_width  = float(pole_3)
species      = pole_4

print(f"Odczytano: {species}, sepal=({sepal_length}, {sepal_width}), petal=({petal_length},
{petal_width})")
```

Generowanie raportu markdown

```
# Trzy eksperymenty ML – każdy jako zestaw osobnych zmiennych
m1_name = "Exp_01"
m1_acc  = 0.85
m1_prec = 0.83
m1_rec  = 0.87

m2_name = "Exp_02"
m2_acc  = 0.89
m2_prec = 0.88
m2_rec  = 0.90

m3_name = "Exp_03"
m3_acc  = 0.87
m3_prec = 0.85
m3_rec  = 0.89

# Wiersze raportu jako lista – potem łączymy przez '\n'.join
wiersze = [
    "# Raport eksperymentów ML",
    "",
    "| Experiment | Accuracy | Precision | Recall |",
```

```
    "|------------|----------|----------|--------|",
    f"| {m1_name:<10} | {m1_acc:>7.2%} | {m1_prec:>8.2%} | {m1_rec:>5.2%} |",
    f"| {m2_name:<10} | {m2_acc:>7.2%} | {m2_prec:>8.2%} | {m2_rec:>5.2%} |",
    f"| {m3_name:<10} | {m3_acc:>7.2%} | {m3_prec:>8.2%} | {m3_rec:>5.2%} |",
    "",
    "_Raport wygenerowany bez użycia Pandas – same listy i stringi._"
]

raport_md = '\n'.join(wiersze)
print(raport_md)

# Zapis do zmiennej, którą potem można zapisać do pliku (zapis plików – w kolejnych lekcjach)
```

Wyjaśnienie: Bez pętli możemy przetwarzać konkretne rekordy, ale pełne iterowanie po całym datasecie wymaga `for`-a (Lekcja 7). Powyższy wzorzec (`split` → `strip` → konwersja typu → formatowanie) jest jednak podstawą, na której zbudujemy późniejsze, bardziej rozbudowane przetwarzanie.

---

8. 🧪 Zadania do samodzielnej pracy

> **Uwaga:** zadania w tej lekcji korzystają wyłącznie z list, stringów, f-stringów i operatorów z Lekcji 1–5. Nie używamy pętli `for` / `while` ani instrukcji `if` — te poznasz w Lekcji 7. Każde zadanie da się rozwiązać za pomocą metod list/stringów i indeksowania.

✏️ Zadania podstawowe (1–8)

1. ✏️ **Zadanie 1 – Tworzenie i indeksowanie listy**

Stwórz listę zawierającą pomiary temperatury z 7 dni: `[22.5, 23.0, 21.5, 24.0, 23.5, 22.0, 23.0]`. Wyświetl:

- temperaturę z pierwszego dnia,
- temperaturę z ostatniego dnia,
- temperatury z trzech środkowych dni (slicing).

(proste)

1. ✏️ **Zadanie 2 – Dodawanie elementów do listy**

Zacznij od pustej listy `results = []`. Dodaj do niej 5 wartości accuracy: 0.75, 0.82, 0.79, 0.85, 0.88 (użyj `append`). Następnie wyświetl:

- długość listy (`len`),
- średnią wartość (`sum / len`),
- maksymalną wartość (`max`),
- minimalną wartość (`min`).

(proste)

1. ✏️ **Zadanie 3 – Sortowanie listy i top-N**

Masz listę wyników: `[0.78, 0.92, 0.85, 0.88, 0.81, 0.95, 0.79]`. Posortuj ją malejąco (`sort(reverse=True)`) i wyświetl 3 najlepsze wyniki, wycinając `[:3]`.

(proste)

1. ✏️ **Zadanie 4 – Czyszczenie pojedynczej nazwy kolumny**

Masz string: `raw_col = " Customer ID "`. Wyczyść go zgodnie z konwencją:

- usuń białe znaki z początku i końca (`strip`),
- zamień na małe litery (`lower`),
- zamień spacje na podkreślniki (`replace`).

Wynik powinien brzmieć: `"customer_id"`.

(proste)

1. ✏️ **Zadanie 5 – Parsowanie linii CSV**

Masz string: `linia = "Alice,28,Data Scientist,75000"`. Użyj `split(',')` i indeksowania, aby wyciągnąć:

- imię (`str`),
- wiek (skonwertowany do `int`),
- zawód (`str`),
- pensję (skonwertowaną do `int`).

Wyświetl informacje w formacie: `"Alice ma 28 lat, pracuje jako Data Scientist i zarabia $75000"`.

(proste)

1. ✏️ **Zadanie 6 – Formatowanie raportu**

Masz dane: `model_name = "XGBoost"`, `accuracy = 0.8934`, `training_time = 45.67`. Użyj f-stringa, aby stworzyć wieloliniowy raport:

```
Model: XGBoost
   Accuracy: 89.34%
   Training time: 45.67s
```

(proste)

1. ✏️ **Zadanie 7 – Zliczanie wystąpień**

Masz listę predykcji: `['spam', 'ham', 'spam', 'spam', 'ham', 'ham', 'spam']`. Użyj metody `count()`, aby policzyć wystąpienia `'spam'` i `'ham'`. Wyświetl wynik jako: `"spam: X, ham: Y"`.

(proste)

1. ✏️ **Zadanie 8 – Ekstrakcja informacji ze ścieżki**

Masz string: `"model_results_2024_Q3.csv"`. Używając slicingu lub `split`, wyciągnij rok (`"2024"`) i kwartał (`"Q3"`). Dla rozszerzenia pliku użyj `endswith` — przypisz do zmiennej wynik `filename.endswith('.csv')` i wyświetl go.

(proste)

✏️ Zadania średnie (9–12)

1. ✏️ **Zadanie 9 – Confusion matrix i metryki**

Masz confusion matrix dla klasyfikacji binarnej:

```
cm = [
       [80, 10],   # TN, FP
       [5, 45]     # FN, TP
   ]
```

Wyciągnij wartości TN, FP, FN, TP przez dwuwymiarowe indeksowanie i oblicz:

- `accuracy = (TP + TN) / (TP + TN + FP + FN)`,
- `precision = TP / (TP + FP)`,

- `recall = TP / (TP + FN)`.

Wyświetl wyniki w formacie procentowym z dwoma miejscami po przecinku.

(średnie)

1. ✏️ **Zadanie 10 – Łączenie i dzielenie listy**

Masz listę nazw modeli: `['LogisticRegression', 'RandomForest', 'XGBoost', 'SVM']`. Zbuduj:

- string `"LogisticRegression, RandomForest, XGBoost, SVM"` (użyj `join`),
- string SQL: `"SELECT model FROM (LogisticRegression, RandomForest, XGBoost, SVM)"` — podstaw złączoną listę w odpowiednie miejsce przez f-string.

Następnie podziel wynikowy string przez `split(', ')` i sprawdź, czy otrzymałaś/eś z powrotem identyczną listę (porównaj operatorem `==` — otrzymasz `True` / `False`, które wyświetl).

(średnie)

1. ✏️ **Zadanie 11 – Parsowanie linii logu**

Masz string logu: `"2024-01-15 10:23:45 ERROR Training failed due to OOM"`. Użyj `split(' ', 3)` (z `maxsplit=3`), aby uzyskać listę 4-elementową: `[data, czas, poziom, wiadomość]`. Wyciągnij poszczególne pola i wyświetl w formacie:

```
[2024-01-15 10:23:45] ERROR: Training failed due to OOM
```

Dodatkowo sprawdź, czy poziom jest jednym z typowych wartości — użyj operatora `in` z listą `['INFO', 'WARNING', 'ERROR', 'DEBUG']` i zapisz wynik (`True` / `False`) do zmiennej `poziom_znany`, którą wyświetlisz.

(średnie)

1. ✏️ **Zadanie 12 – Budowanie zapytania SQL**

Masz tabelę `"customers"`, listę kolumn `['id', 'name', 'email', 'city']` oraz warunek `"city = 'New York'"`. Użyj `', '.join(kolumny)` i f-stringa, aby zbudować zapytanie:

```
SELECT id, name, email, city FROM customers WHERE city = 'New York'
```

(średnie)

🧠 Zadania wyzwanie (13–18)

1. 🧠 **Zadanie 13 – Ranking modeli bez pętli**

Masz 4 modele jako osobne zmienne:

```
m1 = ['LogisticRegression', 0.78]
    m2 = ['RandomForest',       0.92]
    m3 = ['SVM',                0.85]
    m4 = ['XGBoost',            0.95]
```

Zbuduj listę `wszystkie = [m1, m2, m3, m4]` (lista list). Ponieważ `sort()` na liście list z argumentami niekompatybilnymi bez dodatkowej wiedzy wymagałby klucza (poznasz w L8), zrób to inaczej: wyciągnij same accuracy do nowej listy (`[m1[1], m2[1], m3[1], m4[1]]`) i zsortuj ją malejąco. Wyświetl top 3 wartości oraz `max` i `min` z listy accuracy.

(challenge)

1. 🧠 **Zadanie 14 – Ekstrakcja metadanych z nazwy pliku**

Masz ścieżkę: `"/data/processed/2024/experiment_42_v3.csv"` . Za pomocą `split('/')` , indeksowania i metod stringów wyciągnij:

- nazwę pliku (`"experiment_42_v3.csv"`),
- katalog (`"/data/processed/2024"`),
- nazwę bez rozszerzenia (`"experiment_42_v3"`),
- rozszerzenie (`".csv"`),
- rok z katalogu (`"2024"`).

Wyświetl wszystko w czytelnym raporcie (f-string z wyrównaniem).

(challenge)

1. 🧠 **Zadanie 15 – Normalizacja pojedynczej kategorii**

Masz dwie wartości kategorii z różnych źródeł:

```
kat_a = "  Electronics  "
    kat_b = "ELECTRONICS"
```

Znormalizuj obie przez `strip().lower()` i porównaj operatorem `==`. Wyświetl wartości przed normalizacją, po normalizacji oraz wynik porównania (`True` lub `False`).

(challenge)

1. 🧠 **Zadanie 16 – Raport w tabeli ASCII**

Masz trzy eksperymenty jako osobne zmienne:

```
e1 = ["Exp_01", 0.85, 1.23]
    e2 = ["Exp_02", 0.89, 4.56]
    e3 = ["Exp_03", 0.87, 7.89]
```

Wyświetl tabelę:

```
==========================================
    Experiment    Accuracy       Time (s)
    ==========================================
    Exp_01           85.00%          1.23
    Exp_02           89.00%          4.56
    Exp_03           87.00%          7.89
    ==========================================
```

Użyj f-stringów z wyrównaniem (`<` , `>`) i szerokością pól. Dla separatora `=` użyj mnożenia stringa: `"=" * 44` .

(challenge)

1. 🧠 **Zadanie 17 – Walidacja wartości z formularza**

Użytkownik wprowadził trzy pola jako stringi:

```
pole_imie    = "  Alice  "
    pole_wiek    = "  28  "
    pole_email   = "  alice@EXAMPLE.com  "
```

Dla każdego pola wykonaj `strip()` (i dodatkowo `lower()` dla emaila). Następnie sprawdź warunki:

- `czy_imie_ok = pole_imie_czyste.isalpha()` ,
- `czy_wiek_ok = pole_wiek_czyste.isdigit()` ,

- `czy_email_ok = '@' in pole_email_czyste` .

Wyświetl każdą wartość po czyszczeniu oraz zbiorczy wynik walidacji w formie: `"Wszystkie pola OK? X"` gdzie `X` to `czy_imie_ok and czy_wiek_ok and czy_email_ok` (operator logiczny z Lekcji 4 — zwraca `bool`).

(challenge)

1. 🧠 **Zadanie 18 – Generowanie raportu markdown**

Masz listę trzech eksperymentów jako osobne zmienne (podobnie jak w zadaniu 16). Zbuduj raport markdown z tabelą:

```
# Raport

    | Experiment | Accuracy | Time |
    |------------|----------|------|
    | Exp_01 | 85.00% | 1.23s |
    | Exp_02 | 89.00% | 4.56s |
    | Exp_03 | 87.00% | 7.89s |
```

Utwórz listę wierszy (`wiersze = [...]`) i złącz je przez `'\n'.join(wiersze)` . Wyświetl wynik.

(challenge)

---

9. 📚 Podsumowanie

W tej lekcji nauczyłeś/aś się:

Kluczowe koncepcje:

1. **Listy** (mutowalne kolekcje):

- tworzenie z `[]` , indeksowanie dodatnie i ujemne, slicing `[start:stop:step]` ,
- dodawanie elementów: `append()` , `extend()` , `insert()` , operatory `+` i `*` ,
- usuwanie: `remove()` , `pop()` , `clear()` , `del` , slicing z pustą listą,
- sortowanie: `sort()` (in-place) i `sorted()` (nowa lista), `reverse()` ,
- przeszukiwanie: `index()` , `count()` , operator `in` / `not in` ,

- wbudowane funkcje: `len()`, `min()`, `max()`, `sum()`,
- listy wielowymiarowe (lista list) jako macierze.

1. **Stringi** (niemutowalne sekwencje znaków):

- tworzenie (pojedyncze, podwójne, potrójne cudzysłowy, raw stringi),
- indeksowanie i slicing identyczne jak dla list,
- konkatenacja `+`, powtarzanie `*`, `len()`, operator `in`,
- transformacje: `strip()`, `upper()`, `lower()`, `title()`, `capitalize()`, `replace()`,
- walidacja: `startswith()`, `endswith()`, `isdigit()`, `isalpha()`, `isalnum()`, `isspace()`,
- parsowanie: `split()`, `join()`, `find()`, `count()`,
- zaawansowane formatowanie f-stringów: wyrównanie, precyzja, separatory, notacja naukowa, padding z zerami.

1. **Różnice listy vs stringi:**

- listy są MUTOWALNE (można modyfikować elementy przez indeks), stringi NIE,
- listy mogą przechowywać mieszane typy, stringi tylko znaki,
- obie struktury wspierają ten sam interfejs sekwencji (indeksowanie, slicing, `len`, `in`).

1. **Zastosowania w Data Science:**

- listy: przechowywanie pomiarów, nazw kolumn, metryk, macierze 2D,
- stringi: parsowanie CSV, czyszczenie nazw kolumn i kategorii, raporty, zapytania SQL.

Aspekt środowiskowy:

- `append()` / `extend()` są efektywniejsze niż operator `+` na listach (brak kopiowania),
- `join()` znacząco szybsze niż konkatenacja `+` dla wielu stringów,
- modyfikacja in-place oszczędza pamięć; slicing jest szybszy od ręcznego kopiowania.

Przygotowanie do następnych lekcji:

- W **Lekcji 6** poznasz kolejne struktury danych: **słowniki**, **zbiory** i **krotki**, które uzupełnią arsenal kolekcji.
- W **Lekcji 7** dodasz **pętle** (`for`, `while`) i **warunki** (`if` / `elif` / `else`) — wtedy będziesz w stanie automatycznie przetwarzać całe listy i stringi, a nie tylko pojedyncze elementy.

- W **Lekcji 8** poznasz **funkcje**, **lambda** oraz **list comprehensions** — eleganckie sposoby na operacje masowe, oparte na dzisiejszych fundamentach.

---

10. 📖 Dodatkowe zasoby

Dokumentacja Python:

- Listy (Lists) — oficjalna dokumentacja.
- Stringi (str methods) — wszystkie metody stringów.
- Format Specification Mini-Language — pełna specyfikacja formatowania f-stringów.

Tutoriale:

- Real Python – Lists and Tuples.
- Real Python – Strings and Character Data.
- Real Python – Python f-Strings.

Praktyka:

- HackerRank – Python Strings.
- LeetCode – Easy Array Problems — wiele rozwiązań korzysta tylko z list.

ROZPOCZNIJ KARIERĘ W DATA SCIENTIST

Gotowy na następny krok?

Ten materiał to tylko początek. Dołącz do kursu Data Scientist Online + AI i zdobądź praktyczne umiejętności pod okiem ekspertów.

- 5,5 miesiąca intensywnej nauki
- Zajęcia na żywo 2x w tygodniu
- Certyfikat + pomoc w znalezieniu pracy

Zobacz szczegóły kursu →

Zadzwoń do nas

info@learnit.com.pl +48 573-580-990 learnit.com.pl